

The Canonical Story method

A reusable format for writing a pitch deck that survives a judge checking it.

Copy this file, fill in the blanks, delete the guidance. Everything here came out of building four applications that drifted apart because each was written from scratch, and then having an outside reviewer find the seams.

How to use this

Write ONE source file. Generate every document from it.

```
STORY.md (this template, filled in)
|
+-- pitch deck
+-- grant application
+-- hackathon submission
+-- README
+-- demo script
+-- investor email
```

The rule that makes it work: **if a document disagrees with the source, the document is wrong. If the source is wrong, fix it there first and propagate.**

Without this, four documents claim four different fee structures and you do not notice until someone reads two of them side by side.

PART 1: THE SOURCE FILE

1. The thesis

[One sentence. The reason the company exists, not a feature.]

Write the sentence you would defend if everything else were deleted.

Test it: does it state a belief someone could disagree with? "We make X easy" is not a thesis, because nobody is arguing for hard. "Nobody should have to own a gas token to get paid" is, because most of the industry currently assumes you do.

The one-liner

[Company] does [what] for [who], so that [outcome], without [the thing everyone assumes is necessary].

Test: a stranger reads it once and repeats it back correctly. That is the bar. Not "sounds impressive", repeatable.

Banned words

Copy this list. Every one is on a judge's red-flag list:

universal, revolutionary, democratizing, redefining, disruptive, seamless, cutting-edge, next-generation, paradigm, empower, leverage, game-changing, world-class, best-in-class

Add your own as you catch yourself. Enforce it in the build (Part 3).

2. The spine

Six beats. Do not add a seventh.

Beat	Yours
Once upon a time	[the world before]
Every day	[the recurring pain, with a number]
Until one day	[what you built]
Because of that	[first consequence]
Because of that	[second consequence]
Until finally	[the world after]

This is Pixar's story spine. It works because it forces causality: each beat must follow from the last. If your "because of that" does not actually follow, the product logic has a hole in it and the spine just found it.

3. The one user

[Name]. [Role]. [One specific situation.]

One person, named. Referenced on the problem slide, the product slide, and in the demo.

Not "individuals, businesses, and enterprises." That is three users, which a judge reads as none. "Building for everyone" is explicitly on the red-flag list.

Who you are NOT for, said out loud when asked: [the adjacent group you deliberately do not serve, and why they are already fine]

Naming who you exclude is the fastest way to prove you have a real user rather than a demographic.

Secondary segments: [list them]. They live in the appendix. They are expansion, not positioning.

On geography and scope

If you started in one place, say so as a starting point, not an identity:

"We started in [place] because it is the hardest corridor, not because it is the limit."

Check whether your own product is already broader than your pitch. Ours was: the code shipped four currencies and four payment rails while the deck still said one region. The narrow framing was not modest, it was **inaccurate**.

4. The hard part

What	How you solved it	Where in the code
[problem 1]	[mechanism]	path/to/file.ts
[problem 2]	[mechanism]	path/to/file.ts

This table is the proof behind the thesis. Every row must point at a real file.

Then write one paragraph on the single hardest item, explaining why it is hard in terms a non-specialist understands. This is usually the strongest page in the whole deck, because it is the one thing a competitor cannot copy from your landing page.

Closing line that works: "Most teams solve one and stop. We solved four."

5. Approved numbers

Only these. If a number is not in this section, it does not go in a document.

Claim	Value	How it was verified
[metric]	[value]	curl https://... returned it on [date]
[metric]	[value]	path/to/file.ts:42

Every number needs a verification column. Not a source, a **verification**: the command or file path someone else could run to confirm it.

Banned numbers

Banned	Why
[old figure]	[what it actually is now]

When you correct a number, record the old one here with the reason. Otherwise it reappears in the next document written from memory.

The trap worth naming. We had a slide reading "1.25%, returned live by our public fee endpoint." The endpoint returned 1%. The one number that advertised its own verifiability was the one that failed it. **If you invite a check, pass it.**

Traction, stated exactly

[The real number, however small.] [One sentence of context.]

Do not inflate. Do not hide. A judge who has seen two hundred decks claiming ten thousand users respects a real small number **provided it is verifiable**.

Framing that works: "Small on purpose. We proved the rail before opening it."

6. Distribution

Surface	Status	Why it matters
[channel]	Live / Built / Planned	[why this one]

Be honest in the status column. A broken surface listed as live is the kind of thing a judge finds in thirty seconds.

7. Why now

Three to four reasons the answer is this year, not three years ago.

1. **[Technical change]** that made the approach possible.
2. **[Market or regulatory change]** that made it viable.
3. **[Behaviour change]** that means no new habit to teach.

If you cannot answer "why now", the honest conclusion is that someone would already have built it. Find the change or reconsider.

8. Proof assets

Asset	Status
[thing a judge can independently verify]	LIVE / MISSING / BLOCKED on [x]

Rank by how checkable they are. One verifiable artifact beats five claims.

The strongest kind is a **cross-link**: two independent public sources that agree. Ours was a registry entry whose owner address matched the address our contract had configured, so registry and contract were provably the same operator. One click for a judge, and very hard to fake.

9. Known weaknesses and honest answers

Weakness	Your answer
[the obvious hole]	[the honest response, 1 to 2 sentences]

Never hide these. **A judge who finds a concealed weakness discounts everything else. A judge who hears you name it first trusts the rest.**

Write the answer before you are asked, so you are not improvising under pressure.

10. Fix before submitting

Done. [struck-through list, so you can see momentum]

Open, in priority order. [ordered by what blocks what, not by effort]

11. The clock

Find the **real** deadline. Ours said "closes 12 October" while the form said "final submission opens 6 October." Six days, discovered by reading a screenshot rather than the announcement.

When	What
Now	[the thing that blocks the most other things]
...	...
Before the deadline	Ask two people outside the team to read it

Submit early. The last 48 hours are for problems you did not predict.

PART 2: THE DECK ITSELF

Slide count

10 slides maximum. Cap it and enforce the cap in the build.

#	Slide	The one job it does
1	Cover	The one-liner, nothing else
2	Problem	One named user, one specific day
3	Product	What it does, in a numbered list
4	Distribution	Where the users already are
5	Why this is hard	The proof table from section 4
6	Traction	Real numbers, plus one verifiable link
7	Business model	How money is made, per stream
8	Competition and moat	Plain rows, see below
9	Compliance / risk	The thing they will worry about
10	Team and ask	Who, and what you want

Slide rules

One idea per slide. If a slide needs "and", it is two slides or one is cut.

5 to 7 ideas in the whole deck. Kevin Hale's number. More than that and nothing is remembered.

Legible, simple, obvious. If a judge has to work to read it, they stop.

No screenshots of interfaces. A bulleted list of the steps beats a screenshot of the screen where the steps happen. Screenshots are illegible at deck size and they show the UI rather than the logic.

No competitor matrix. The grid where you have all the ticks is on every red-flag list, because everyone's grid shows them winning. Use plain rows: what others do, what you do, why the difference is structural.

Assume it is a slide on a dating app. Two seconds, then swipe. What survives two seconds?

You do not win by impressing judges. You win by not giving them a reason to reject you.

PART 3: BUILD GUARDS

The part most people skip, and the reason decks decay.

A convention nobody enforces decays one edit at a time. Make the build fail.

If you generate your deck from code (python-pptx, Marp, Slidev), add assertions. If you write it by hand, run these as a checklist script against the exported text.

```
# Run against the rendered text of every slide.

blob = extract_all_text(deck)

# 1. Typography you have banned
for ch, name in ((chr(0x2014), "em dash"), (chr(0x2013), "en dash")):
    if ch in blob:
        raise SystemExit(f"BUILD FAILED: {name} present.")

# 2. Buzzwords and vanity metrics
BANNED = ["universal", "revolutionary", "democratiz", "disruptive",
          "seamless", "cutting-edge", "next-generation", "paradigm",
          "100% passing", "world-class"]
for term in BANNED:
    if term.lower() in blob.lower():
        raise SystemExit(f"BUILD FAILED: banned term '{term}'.")

# 3. Numbers that must match a live source
#
# Verified YYYY-MM-DD: curl https://api.example.com/fees returns "1"
# If the live value changes, change BOTH this constant and the slide.
LIVE_FEE = "1%"
if LIVE_FEE not in blob:
    raise SystemExit(f"BUILD FAILED: {LIVE_FEE} not found. "
                    "The live endpoint and the deck must agree.")

# 4. Slide cap
if slide_count(deck) > 10:
    raise SystemExit("BUILD FAILED: more than 10 slides.")
```

Three lessons about guards

A guard that cannot fail is decoration. Test every guard by deliberately breaking the thing it protects, confirming the build fails, then restoring. Ours passed for a week while matching nothing.

A guard that always fails gets disabled. Our first version matched its own source file, which necessarily names the banned terms in order to forbid them. It failed on every run including a clean tree, which makes it useless. Exclude the guard file itself.

Watch for false positives. A naive search for a banned fee figure matched `netMarginBeforeCacUsd`. Require word boundaries, and never let a guard cry wolf on unrelated content.

PART 4: THE VERIFICATION PASS

Do this once, before submitting. Budget two hours.

1. **Enumerate every claim** in the deck. Every number, every capability, every integration. One per line.
2. **Against each, write the verification:** a URL to curl, a file path with a line number, or a transaction hash.
3. **Actually run them.** Not from memory.
4. **Anything you cannot verify comes out.** Not softened, out.

This is how we found a published contract address with **zero bytecode** sitting in three developer documents, one of them inside a copy-pasteable snippet. It had been there for weeks, it looked plausible because its first six characters matched the real address, and anyone following the docs would have sent funds to an address with no code.

Nobody had checked, because everyone assumed someone had.

THE CHECKLIST

Before you submit:

- [] One source file exists and every document derives from it
- [] The one-liner passes the stranger test
- [] The spine has exactly six beats and each follows from the last
- [] One named user, and a stated non-user
- [] Every number has a verification command or file path
- [] Banned numbers are recorded with what they actually are
- [] Traction is stated exactly, unflattering and true
- [] Weaknesses are written down with answers
- [] 10 slides or fewer, one idea each
- [] No competitor matrix, no interface screenshots
- [] Build guards exist AND have been tested by deliberately breaking them

- [] Every claim verified against a live source in one sitting
 - [] Read by two people outside the team
 - [] Submitted early
-

The one principle

Everything here reduces to one thing:

Write only what someone else could check, then check it yourself first.

A small verified number beats a large unverifiable one. A named weakness beats a hidden one. A claim with a curl command beside it is worth more than a paragraph of adjectives.